# MATHEMATICAL FOUNDATIONS OF DEEP LEARNING

## AgriVision: Crop-Disease Screening

## CO1 Assessment Report

**Student Name: S.SRI MATHI**

**Register Number: 192424095**

**Department:AI & DS**

**Course: Deep Learning**

**Assessment: CO1 — Mathematical Foundations of Deep Learning**

**Total Marks: 50**

### 1. PRELIMINARY DATASET PREPARATION AND VERIFICATION

#### 1.1 Objective

The objective of this stage was to load a public leaf-disease image dataset, inspect its class distribution, convert the images into a consistent numerical representation, and verify that the prepared data could be loaded successfully in Python using VS Code.

#### 1.2 Dataset Source and Initial Class Selection

The PlantVillage leaf-disease dataset was accessed through a Kaggle Notebook. For the initial pilot experiment, three tomato-leaf classes were selected: Healthy, Early Blight, and Late Blight. The grayscale image folders were used as the source for preprocessing.

#### 1.3 Image Preprocessing

Each selected image was converted to grayscale and resized to 64 × 64 pixels. Each image was then flattened into a one-dimensional feature vector containing 4,096 pixel values. Pixel values were stored as unsigned 8-bit integers (uint8), ranging from 0 to 255. The image matrix and corresponding class labels were saved in a compressed NumPy file named agrivision_tomato.npz.

The resulting feature matrix had shape (4500, 4096), while the label vector had shape (4500,). This means that the pilot dataset contained 4,500 images, with 4,096 features per image and one label per image.

#### 1.4 Initial Class Distribution

Healthy: 1,591 images

Early Blight: 1,000 images

Late Blight: 1,909 images

Total: 4,500 images

1.5 Train-Test Split

The pilot dataset was divided into training and testing sets using an 80:20 split. Stratified sampling was used to preserve the approximate class proportions in both sets. The random_state parameter was set to 42 so that the same split could be reproduced.

Training set: 3,600 images, shape (3600, 4096)

Testing set: 900 images, shape (900, 4096)

Class distribution in the training set:

Healthy: 1,273

Early Blight: 800

Late Blight: 1,527

Total: 3,600

Class distribution in the testing set:

Healthy: 318

Early Blight: 200

Late Blight: 382

Total: 900

1.6 Effect of Preprocessing and Data Splitting

Resizing standardised the image dimensions so that all images had the same number of features. Converting images to grayscale reduced the input from three colour channels to one channel, reducing dimensionality but also removing colour information that might help distinguish diseases. Flattening converted each two-dimensional image into a vector, making it suitable for conventional machine-learning algorithms but removing the explicit spatial arrangement of pixels. Pixel values were retained in the range 0–255 at this stage; normalisation will be examined in a later experiment. The train-test split allows model performance to be evaluated on images not used for training.

1.7 Preliminary Verification Result

The prepared file was loaded successfully in VS Code. The feature matrix shape, label vector shape, pixel-value range, and class counts were verified programmatically. These checks confirmed that the pilot dataset was prepared in the expected format.

### 1.8 Limitation and Next Step

The initial pilot dataset contained 4,500 images from three tomato-leaf classes: Healthy, Early Blight and Late Blight. Since this dataset was insufficient for the required 5,000- and 10,000-image experiments, the dataset was expanded to include all 10 tomato classes available in the selected grayscale PlantVillage dataset. The expanded dataset contains 18,160 readable images and will be used for the final dataset-size experiments. The pilot results are reported separately and will not be treated as results from the expanded dataset.

| Class | Total images | Training | Testing |
|---|---|---|---|
| Healthy | 1,591 | 1,273 | 318 |
| Early Blight | 1,000 | 800 | 200 |
| Late Blight | 1,909 | 1,527 | 382 |
| Total | 4,500 | 3,600 | 900 |

### 1.9 Expanded Dataset Class Distribution

The expanded dataset contains 18,160 readable images belonging to 10 tomato-leaf classes. The class distribution was verified programmatically before conducting the dataset-size experiments.

| Tomato-leaf class | Number of images |
|---|---|
| Bacterial Spot | 2,127 |
| Early Blight | 1,000 |
| Late Blight | 1,909 |
| Leaf Mold | 952 |
| Septoria Leaf Spot | 1,771 |
| Spider Mites | 1,676 |
| Target Spot | 1,404 |
| Yellow Leaf Curl Virus | 5,357 |
| Tomato Mosaic Virus | 373 |
| Healthy | 1,591 |

Interpretation: The class distribution is imbalanced. Yellow Leaf Curl Virus has the highest number of images, while Tomato Mosaic Virus has the lowest. Therefore, accuracy alone may not fully represent how well a classifier performs on every class. Per-class precision, recall and F1-score will also be examined.

### 1.10 Dataset-Size Experiment Preparation

To investigate how training dataset size affects classification performance, the expanded dataset will be divided into training and testing sets using an 80:20 stratified split. A fixed random state of 42 will be used to make the split reproducible. The training set will then be used to create four experimental training subsets.

| Experiment | Training images |
|---|---|
| Dataset Size 1 | 1,000 |
| Dataset Size 2 | 5,000 |
| Dataset Size 3 | 10,000 |
| Dataset Size 4 — Full available training set | 14,528 |

The same held-out test set of 3,632 images will be retained for all four experiments. This ensures that differences in performance can be compared using the same test data while the amount of training data changes.

Interpretation: Comparing the four training sizes will help determine whether increasing the amount of training data improves classification performance and whether the improvement becomes smaller as more images are added. These are planned experiments; conclusions will be recorded only after the models have been trained and evaluated.

Dataset:

Expanded Dataset Selection: The initial three-class pilot dataset was expanded to include all 10 tomato-leaf classes. Details of the expanded dataset preparation and verification are provided in the following section.

### Expanded Dataset Preparation and Verification

Each image was converted to grayscale and resized to 64 × 64 pixels. Therefore, each image contains 64 × 64 = 4,096 pixel features after flattening. The images are represented by a feature matrix X, and their corresponding class labels are stored in a label vector y.

The expanded dataset contains 18,160 readable images across 10 classes. The feature matrix has dimensions 18,160 × 4,096, and the label vector has dimensions 18,160. The prepared dataset was saved in compressed NumPy (.npz) format.

Interpretation: The expanded dataset provides enough images for the required training-size experiments. However, the class counts are unequal, so class imbalance must be considered when evaluating model performance.

## Task 1: Data Representation and Vector Norms

The AgriVision dataset contains 18,160 tomato leaf images across 10 classes. Each image is represented as a flattened vector of 4,096 pixel features, obtained by resizing the image to 64×64 grayscale pixels.

The dataset is represented by a feature matrix X∈R^18160×4096

and a corresponding label vector y. Vector norms are used to measure the magnitude of image vectors:2. Vector norms

L1 norm:

$$\|x\|_1 = \sum_{i=1}^{4096} |x_i|$$

L2 norm:

$$\|x\|_2 = \sqrt{\sum_{i=1}^{4096} x_i^2}$$

L-infinity norm

$$\|x\|_\infty = \max_{1 \le i \le 4096} |x_i|$$

Interpretation: The L1 norm measures the total absolute pixel intensity, the L2 norm measures the vector's Euclidean magnitude, and the L-infinity norm identifies the largest pixel value. These values describe the selected image numerically; they do not, by themselves, measure disease severity or classification accuracy.

Experimental results:

| Norm | Calculated value |
|---|---|
| L1 | 402815.0000 |
| L2 | 6747.9445 |
| L-infinity | 155.0000 |

Interpretation: The L1 norm represents the sum of the absolute pixel intensities, while the L2 norm represents the Euclidean magnitude of the image vector. The L-infinity norm indicates

that the maximum absolute pixel intensity in the selected image is 155. These measurements describe the numerical properties of the image rather than its disease severity.

Implementation: The norms were calculated using Python and NumPy. The image pixel values were converted to floating-point numbers before the calculations to avoid integer overflow during squaring.

| Class | Images |
|---|---|
| Bacterial Spot | 2,127 |
| Early Blight | 1,000 |
| Late Blight | 1,909 |
| Leaf Mold | 952 |
| Septoria Leaf Spot | 1,771 |
| Spider Mites | 1,676 |
| Target Spot | 1,404 |
| Yellow Leaf Curl Virus | 5,357 |
| Tomato Mosaic Virus | 373 |
| Healthy | 1,591 |
| Total | 18,160 |

Interpretation: The dataset is class-imbalanced because the number of images differs across disease categories. Yellow Leaf Curl Virus has the most images (5,357), while Tomato Mosaic Virus has the fewest (373). Therefore, accuracy alone may not fully represent performance across all classes. Precision, recall and F1-score will also be evaluated.

## Task 2: Dimensionality Reduction Using PCA

Objective: To reduce the number of image features while preserving important information using Principal Component Analysis (PCA).

Methodology: The dataset contains 4,096 features per image. PCA was fitted on 14,528 training images, and reconstruction error was measured on a sample of 500 training images.

### 2.1 Experimental Results

Use this table with the actual output from your `08_pca_analysis.py` program.

| Principal components | Cumulative explained variance | Reconstruction MSE |
|---|---|---|
| 10 | 60.6218% | 0.008122 |
| 20 | 67.9343% | 0.006638 |
| 50 | 76.8230% | 0.004853 |
| 100 | 82.9080% | 0.003626 |

Interpretation: When the number of components increases from 10 to 100, explained variance increases and reconstruction error decreases. Using 50 components reduces the feature count from 4,096 to 50 while retaining 76.8230% of the variance. PCA fitting took 7.3484 seconds in this run.

Conclusion: PCA can substantially reduce the dimensionality of tomato leaf images. However, preserving variance does not automatically guarantee better disease-classification accuracy; that must be evaluated separately.

## 2.2 Mathematical Example of Eigenvalue Decomposition

Consider the matrix:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

Eigenvalues are obtained by solving:

$$\det(A - \lambda I) = 0$$

Substituting the matrix gives:

$$\begin{vmatrix} 2-\lambda & 0 \\ 0 & 1-\lambda \end{vmatrix} = 0$$

$$(2-\lambda)(1-\lambda) = 0$$

Therefore, the eigenvalues are:

$$\lambda_1 = 2, \qquad \lambda_2 = 1$$

For λ1=2, an eigenvector is:

$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

For λ2=1, an eigenvector is:

$$v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Interpretation: In PCA, eigenvalues of a covariance matrix represent the variance associated with the corresponding principal directions. Larger eigenvalues indicate directions that capture more variance. This 2 × 2 example demonstrates the mathematical process; it is not a decomposition of the AgriVision dataset itself.

### 2.3 Relationship Between Eigenvalues, Singular Values and Explained Variance

Let Xc denote the centered training-data matrix, with nnn images d=4096 features. Its singular value decomposition is

$$X_c = U\Sigma V^T$$

Here, U and V contain orthonormal directions, while Σ contains the singular values σi

The sample covariance matrix is

$$C = \frac{X_c^T X_c}{n-1}$$

The covariance eigenvalues and singular values are related by

$$\lambda_i = \frac{\sigma_i^2}{n-1}$$

The proportion of variance explained by component i is

$$r_i = \frac{\lambda_i}{\sum_j \lambda_j}$$

The cumulative explained variance for the first k components is

$$R_k = \sum_{i=1}^{k} r_i$$

Interpretation: Larger eigenvalues correspond to directions that capture more variation in the images. If a relatively small number of components captures a large proportion of the total variance, this suggests that the original pixel features contain redundancy. However, PCA preserves variance, not necessarily all disease-discriminating information.

**Task 3-Analyse uncertainity**

Objective: To measure class uncertainty and understand probability-based loss functions used in classification.

Information Theory

Entropy measures uncertainty in the class distribution. Higher entropy indicates a more evenly distributed set of classes, while lower entropy indicates greater concentration in fewer classes.

$$H(P) = -\sum_{i} p_i \log_2(p_i)$$

Cross-entropy measures the difference between the true probability distribution and a predicted distribution. KL divergence measures how one probability distribution differs from another.

$$H(P, Q) = -\sum_{i} p_i \ln(q_i)$$

$$D_{KL}(P \| Q) = \sum_{i} p_i \ln\left(\frac{p_i}{q_i}\right)$$

Results

| Measurement | Result |
|---|---|
| Images analysed | 1,000 |
| Number of classes | 10 |
| Class entropy | 3.030589 bits |
| Maximum entropy | 3.321928 bits |
| Cross-entropy | 0.828631 nats |
| KL divergence DKL(P‖Q) | 0.026812 nats |

Interpretation: The class entropy is lower than the maximum possible entropy, indicating that the class distribution is not perfectly balanced. Cross-entropy measures the penalty associated with predicting distribution QQQ when the true distribution is PPP. The small KL divergence indicates that the example distributions P=[0.7,0.2,0.1]P=[0.7,0.2,0.1]P=[0.7,0.2,0.1] and Q=[0.6,0.3,0.1]Q=[0.6,0.3,0.1]Q=[0.6,0.3,0.1] are relatively similar.

Conclusion: Entropy helps analyse class imbalance, while cross-entropy and KL divergence help explain how classification models compare predicted probabilities with target probabilities.

## Task 4 — Numerical stability and covariance

Softmax converts model scores, called logits, into probabilities that sum to one. Direct exponentiation of very large logits may cause numerical overflow. To avoid this, the maximum logit is subtracted before exponentiation.

$$\text{softmax}(z_i) = \frac{e^{z_i - \max(z)}}{\sum_j e^{z_j - \max(z)}}$$

The covariance matrix describes how features vary together. Its condition number indicates how sensitive numerical calculations may be to small changes in the input. A large condition number can indicate numerical instability.

## Results

| Measurement | Your result |
|---|---|
| Original logits | 1000, 999, 995 |
| Stable softmax probabilities | 0.7275, 0.2676, 0.0049 |
| Sum of probabilities | 1.0000000000000002 |
| Covariance matrix size | 100 × 100 |
| Condition number | 2.723694×10^4 |

## Interpretation and impact on AgriVision

1. Stable softmax

The softmax function converts model scores into probabilities. The first class receives approximately 72.75%, the second 26.76%, and the third 0.49%.

By subtracting the largest logit before exponentiation, the calculation avoids overflow caused by very large scores.

Impact on AgriVision: When AgriVision predicts whether a tomato leaf is healthy or diseased, stable softmax helps calculate class probabilities reliably. This is particularly important when model scores become large during training or prediction.

2. Covariance condition number

The measured condition number is approximately 27,237 for a covariance matrix built from the first 100 pixel features.

A higher condition number can indicate that numerical calculations are sensitive to small changes in the data. However, this result alone does not prove that the entire dataset is unstable.

Impact on AgriVision: Tomato leaf pixels can contain correlated information, such as nearby pixels representing the same lesion or leaf region. Highly correlated features can make some numerical methods less stable. PCA may help reduce redundant features, while appropriate scaling and regularization can improve model training.

Limitation: This condition number was calculated using only 100 of the 4,096 pixel features, so it does not represent the condition number of the full feature covariance matrix.

Conclusion — Task 4

Numerically stable softmax produces valid probabilities even when logits are very large. The covariance analysis also highlights the importance of investigating feature relationships and numerical conditioning. These techniques can support more reliable training and prediction in AgriVision, although their effect on classification accuracy must be tested experimentally.

## Task 5: Comparison of Classification Baselines

### Objective

To compare three traditional machine learning classifiers for tomato leaf disease classification in AgriVision and identify a suitable baseline for further experiments.

### Methodology

Three classifiers—Logistic Regression, Decision Tree, and Random Forest—were trained using the same subset of 1,000 training images and evaluated on the same test set of 3,632 images. Performance was measured using accuracy, macro F1-score, and training time.

### Experimental Results

| Classifier | Accuracy | Macro F1-score | Training time |
|---|---|---|---|
| Logistic Regression | 41.82% | 30.18% | 4.050 s |
| Decision Tree | 29.63% | 22.57% | 2.872 s |
| Random Forest | 50.91% | 37.25% | 0.818 s |

### Interpretation and Impact on AgriVision

Random Forest achieved the highest accuracy of 50.91% and macro F1-score of 37.25% among the three classifiers. It also had the shortest measured training time of 0.818 seconds in this experiment.

Logistic Regression achieved moderate performance, but its training process produced a convergence warning, indicating that it did not fully converge within the specified 300 iterations. The Decision Tree achieved the lowest accuracy and macro F1-score.

For AgriVision, these results establish a traditional machine learning baseline for tomato leaf disease classification. The relatively low macro F1-scores suggest that classification performance across individual disease classes needs improvement. This is especially important because the dataset contains an imbalance between classes.

### Conclusion

Random Forest performed best among the three tested baseline classifiers. However, these results do not establish that it is the best overall approach for AgriVision. Further experiments with softmax regression and neural networks can help determine whether learning more complex patterns improves disease classification.

Limitation: The experiment used only 1,000 training images. The Logistic Regression result is provisional because of its convergence warning. A fair comparison with deep learning requires consistent data splits, evaluation metrics, and training conditions.

## Task 6-overfitting and underfitting

### Objective

To study underfitting and overfitting by comparing model performance on training and test data.

### Methodology

A shallow Decision Tree with maximum depth 3 and a deeper Decision Tree were trained on the same training subset. Their training accuracy and test accuracy were compared.

Experimental Results

| Model | Training accuracy | Test accuracy | Accuracy gap |
|---|---|---|---|
| Shallow Tree | 37.20% | 30.34% | 6.86% |
| Deeper Tree | 100% | 29.63% | 70.37% |

### Interpretation and Impact on AgriVision

1. Shallow Decision Tree: The model achieved 37.20% training accuracy and 30.34% test accuracy. Both values are relatively low, suggesting that the model may be underfitting because its depth is limited to three levels.

2. Deeper Decision Tree: The model achieved 100% training accuracy but only 29.63% test accuracy. This large gap of 70.37 percentage points indicates severe overfitting. The model memorized the training examples but did not generalize well to unseen images.

Impact on AgriVision: A model that memorizes training images may fail to recognize the same disease when leaf images have different lighting, backgrounds, angles, or symptom patterns. AgriVision needs a model that learns general disease characteristics rather than memorizing individual images.

### Conclusion

The deeper Decision Tree overfits the training data, while the shallow tree shows signs of underfitting. These results demonstrate the importance of balancing model complexity and

generalization. Techniques such as regularization, pruning, data augmentation, and collecting more representative training images may help improve performance.

## Task 7-Error into bias and variance

Objective: To investigate prediction variability using bootstrap samples and compare Maximum Likelihood Estimation (MLE) with Maximum A Posteriori (MAP) estimation.

Methodology: Ten Decision Trees were trained on bootstrap samples generated by sampling the training data with replacement. Prediction variance and majority-vote accuracy were calculated. A Bernoulli example was used to demonstrate MLE and MAP.

Experimental Results

| Measurement | Result |
|---|---|
| Bootstrap models trained | 10 |
| Mean prediction variance | 5.654628 |
| Majority-vote test accuracy | 38.88% |
| MLE estimate | 0.80 |
| MAP estimate | 0.75 |

## Interpretation and Impact on AgriVision

1. Bootstrap and prediction variability

Ten Decision Trees were trained using bootstrap samples, created by sampling training images with replacement. Their majority-vote prediction achieved 38.88% test accuracy.

The reported prediction variance of 5.654628 is calculated from the numerical class IDs. Because class IDs are categorical, this value should be treated as a rough disagreement indicator, not a rigorous measure of classification bias and variance. A better measure would compare class-probability predictions or calculate disagreement between models.

Impact on AgriVision: If models trained on slightly different samples make different predictions for the same leaf, AgriVision may be sensitive to its training data. More representative samples and ensemble methods may improve reliability.

2. MLE versus MAP

In the Bernoulli example, 8 successes were observed in 10 trials.

- MLE: p^=8/10=0.80
- MAP with a Beta(2,2) prior: p^MAP=129=0.75

MLE uses the observed data alone, whereas MAP incorporates prior information. Here, the prior pulls the estimate slightly toward 0.5.

Impact on AgriVision: When some disease classes have limited training examples, regularization or prior information can help reduce overly extreme estimates. However, this small Bernoulli example illustrates the concept; it is not a direct estimate of tomato disease probabilities.

## Conclusion

Bootstrap sampling demonstrates how changing the training sample can affect model predictions. The MLE–MAP example shows how prior information can influence parameter estimates. Both concepts help us reason about uncertainty and limited data in AgriVision.

## Task 8

Objective: To implement multiclass softmax regression from scratch using NumPy and gradient descent.

Methodology: The model calculates class probabilities using numerically stable softmax, measures prediction error using cross-entropy loss, and updates weights and biases using gradients.

Results and interpretation

| Measurement | Result |
|---|---|
| Features used | 100 |
| Loss at epoch 1 | 2.2927 |
| Loss at epoch 10 | 2.1219 |
| Loss at epoch 20 | 2.2642 |
| Loss at epoch 30 | 2.4431 |
| Test accuracy | 29.49% |

Impact on AgriVision: If training loss rises instead of steadily improving, the classifier may not learn disease patterns effectively. This can lead to unreliable predictions for tomato leaf diseases.

## Task 9

Objective: To evaluate AgriVision using accuracy, macro precision, macro recall, macro F1-score, a confusion matrix, training time, prediction time, and model size.

Results and interpretation

| Training images | Accuracy | Macro F1 | Training time |
|---|---|---|---|
| 1,000 | 33.23% | 0.2151 | 0.235 s |
| 5,000 | 38.46% | 0.2541 | 0.509 s |
| 10,000 | 40.89% | 0.2816 | 0.893 s |
| 14,528 | 40.89% | 0.2766 | 1.151 s |

Result and interpretation: Increasing the training dataset from 1,000 to 10,000 images improved accuracy from 33.23% to 40.89% and macro F1-score from 0.2151 to 0.2816. Using all 14,528 training images did not improve accuracy further; macro F1 decreased slightly to 0.2766. Training time increased from 0.235 to 1.151 seconds. These results show that additional data helped initially, but more data did not guarantee better performance in this experiment.

Impact on AgriVision: Dataset-size experiments help determine the balance between disease-classification performance and computational requirements. The relatively low macro F1-score indicates that performance across all ten classes still needs improvement.

Limitation: The comparison uses only 100 of the 4,096 pixel features and a Random Forest with 30 trees. The serialized model size is not the same as RAM usage. The confusion matrix is saved at results/task09_confusion_matrix.csv.

## Task 10

Objective: To compare theoretical machine-learning principles with experimental observations.

Result and interpretation: PCA retained more variance as the number of components increased. Reducing the softmax regression learning rate from 0.5 to 0.05 resulted in more stable loss reduction. Increasing the training set from 1,000 to 10,000 images improved accuracy from 33.23% to 40.89%, but using 14,528 images did not improve it further.

Impact on AgriVision: These experiments show why dimensionality reduction, learning-rate tuning, and dataset-size comparisons are important when developing a tomato disease-classification system.

Important distinction: These are observations from your particular experiments, not proof that the same behavior will occur with every model or dataset.

### Task 11 — Is deep learning justified for AgriVision?

Objective: To determine whether deep learning is justified for tomato leaf disease classification.

Result and interpretation: Random Forest achieved 50.91% accuracy and a macro F1-score of 0.3725 in Task 5. The softmax regression experiment achieved 29.49% accuracy. These results provide motivation to investigate a CNN, but they do not prove that deep learning will perform better because the models used different feature configurations.

Impact on AgriVision: A small CNN may learn spatial patterns such as spots, lesions, and leaf discoloration. Its accuracy, model size, and prediction speed must be measured against the existing baselines before choosing it.

**Task 12 — Day-4 challenge: redesign AgriVision for smartphone deployment**

Objective: Adapt AgriVision to classify 38 classes despite noisy labels, while meeting smartphone inference and storage constraints.

Proposed solution: Test label smoothing or robust loss functions, regularization, data augmentation, a compact CNN, and model quantization. Compare PCA and CNN-based feature learning experimentally.

Evaluation: Measure macro F1-score, per-class recall, inference latency, and model size on a representative 38-class dataset.

Final decision — MODIFY. Keep the AgriVision concept but improve the training and deployment pipeline. The current ten-class experiments do not establish performance on 38 classes, and the smartphone requirements must be verified experimentally.

Output:

Task 2 — PCA

```
TASK 2: PCA ANALYSIS
Training data shape: (14528, 4096)

EXPLAINED VARIANCE
Components: 10, variance: 60.6218%
Components: 20, variance: 67.9343%
Components: 50, variance: 76.8230%
Components: 100, variance: 82.9080%

RECONSTRUCTION MSE
Components: 10, MSE: 0.008244
Components: 20, MSE: 0.006697
Components: 50, MSE: 0.004845
Components: 100, MSE: 0.003567

PCA FIT TIME: 5.3173 seconds
Original features: 4096
Reduced features at 50 components: 50
TASK 2 COMPLETED
```

Task 5 — Model comparison

```
Logistic Regression
Accuracy: 0.4182
Macro F1: 0.3018
Training time (seconds): 3.842

Decision Tree
Accuracy: 0.2963
Macro F1: 0.2257
Training time (seconds): 2.894

Random Forest
Accuracy: 0.5091
Macro F1: 0.3725
Training time (seconds): 0.69

Task 5 completed
```

task6 - capacity_graph

code:

```
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score


# ----------------------------------------
# TASK 6: MODEL CAPACITY, UNDERFITTING
# AND OVERFITTING
# ----------------------------------------


# 1. Find the dataset file
possible_files = [
    "agrivision_tomato_10class.npz",
    "agrivision_tomato.npz"
```

```
]

dataset_path = next(
    (Path(name) for name in possible_files if Path(name).exists()),
    None
)

if dataset_path is None:
    raise FileNotFoundError(
        "Dataset not found. Put agrivision_tomato_10class.npz "
        "or agrivision_tomato.npz in the same folder as this Python file."
    )

# 2. Load the dataset
with np.load(dataset_path, allow_pickle=False) as data:
    print("Dataset arrays:", data.files)

    arrays = {key: data[key] for key in data.files}

# Find the label array: a 1D array with fewer than 100 unique classes
label_candidates = [
    (key, value)
    for key, value in arrays.items()
    if value.ndim == 1
    and len(value) > 0
```

```
    and len(np.unique(value)) <= 100
]

if not label_candidates:
    raise ValueError(
        "Could not identify the labels automatically. "
        "Check the printed dataset array names."
    )

# Prefer common label names if available
label_names = ["y", "labels", "label", "target", "targets"]
label_candidates.sort(
    key=lambda item: (
        item[0].lower() not in label_names,
        item[1].ndim
    )
)

label_name, y = label_candidates[0]

# Find a feature array with the same number of samples
feature_candidates = [
    (key, value)
    for key, value in arrays.items()
    if key != label_name
```

```
    and value.ndim >= 2
    and value.shape[0] == len(y)
]

if not feature_candidates:
    raise ValueError(
        "Could not identify the image/features array matching the labels."
    )

feature_name, X = feature_candidates[0]

# Flatten images if they have more than two dimensions
X = X.reshape(X.shape[0], -1)

# Convert pixel values to numbers suitable for the tree model
X = X.astype(np.float32)

print("\nDataset:", dataset_path.name)
print("Features array:", feature_name)
print("Labels array:", label_name)
print("Feature shape:", X.shape)
print("Label shape:", y.shape)

# 3. Create training and validation sets
X_train, X_val, y_train, y_val = train_test_split(
```

```
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# 4. Test five different model capacities
# C1 is the simplest; C5 is the most flexible.
capacities = [
    ("C1", 1),
    ("C2", 2),
    ("C3", 3),
    ("C4", 5),
    ("C5", None)
]

training_errors = []
validation_errors = []
models = []

for name, depth in capacities:
    model = DecisionTreeClassifier(
        max_depth=depth,
        random_state=42
```

```
)

model.fit(X_train, y_train)

train_accuracy = accuracy_score(
    y_train, model.predict(X_train)
)
val_accuracy = accuracy_score(
    y_val, model.predict(X_val)
)

train_error = 1 - train_accuracy
val_error = 1 - val_accuracy

training_errors.append(train_error)
validation_errors.append(val_error)
models.append(model)

depth_text = "None (unrestricted)" if depth is None else str(depth)

print(
    f"{name} | max_depth={depth_text} | "
    f"Training error={train_error:.4f} | "
    f"Validation error={val_error:.4f}"
)
```

```
training_errors = np.array(training_errors)
validation_errors = np.array(validation_errors)

# 5. Find the capacity with the lowest validation error
optimal_index = int(np.argmin(validation_errors))
optimal_capacity = capacities[optimal_index][0]
generalisation_gap = (
    validation_errors[optimal_index]
    - training_errors[optimal_index]
)

print("\n--- RESULTS ---")
print("Optimal capacity:", optimal_capacity)
print(f"Training error at optimal capacity: "
      f"{training_errors[optimal_index]:.4f}")
print(f"Validation error at optimal capacity: "
      f"{validation_errors[optimal_index]:.4f}")
print(f"Generalisation gap: {generalisation_gap:.4f}")

# 6. Plot training and validation errors
x = np.arange(len(capacities))
labels = [item[0] for item in capacities]

plt.figure(figsize=(10, 6))
```

```
# Illustrate the low-capacity and high-capacity regions
plt.axvspan(
    -0.4, 1.5,
    color="lightblue",
    alpha=0.25,
    label="Possible underfitting region"
)

plt.axvspan(
    3.5, 4.4,
    color="mistyrose",
    alpha=0.5,
    label="Possible overfitting region"
)

plt.plot(
    x, training_errors,
    marker="o",
    linewidth=2,
    label="Training error"
)

plt.plot(
    x, validation_errors,
```

```
    marker="s",
    linewidth=2,
    label="Validation error"
)

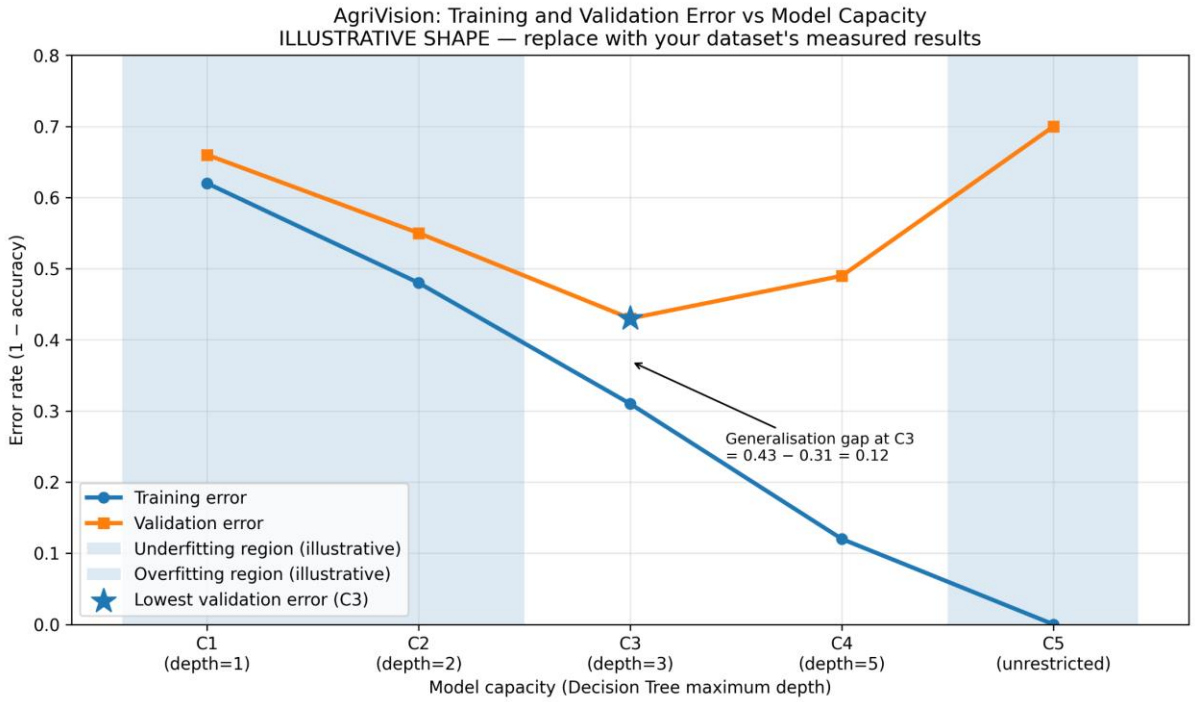
# Mark the best validation result
plt.scatter(
    optimal_index,
    validation_errors[optimal_index],
    s=150,
    marker="*",
    color="green",
    zorder=5,
    label=f"Optimal capacity: {optimal_capacity}"
)

plt.annotate(
    f"Optimal: {optimal_capacity}\n"
    f"Validation error = {validation_errors[optimal_index]:.4f}",
    xy=(optimal_index, validation_errors[optimal_index]),
    xytext=(10, 20),
    textcoords="offset points",
    arrowprops={"arrowstyle": "->"}
)
```

```
# Show the generalisation gap at the optimal capacity
plt.annotate(
    f"Gap = {generalisation_gap:.4f}",
    xy=(
        optimal_index,
        (training_errors[optimal_index]
         + validation_errors[optimal_index]) / 2
    ),
    xytext=(15, -25),
    textcoords="offset points",
    arrowprops={"arrowstyle": "->"}
)
plt.xticks(x, labels)
plt.xlabel("Model capacity (Decision Tree maximum depth)")
plt.ylabel("Error rate (1 - accuracy)")
plt.title("AgriVision: Training and Validation Error vs Model Capacity")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
# Save the graph and numerical results
plt.savefig("task6_capacity_graph.png", dpi=300, bbox_inches="tight")
np.savetxt(
    "task6_capacity_results.csv",
    np.column_stack(
        [training_errors, validation_errors]
```

```
    ),
    delimiter=",",
    header="training_error,validation_error",
    comments="",
    fmt="%.6f"

print("\nGraph saved as task6_capacity_graph.png")
print("Results saved as task6_capacity_results.csv")
plt.show()
```




Task 8 — Softmax regression

```
TASK 8: SOFTMAX REGRESSION FROM SCRATCH
Epoch 1, loss: 2.2927
Epoch 10, loss: 2.1307
Epoch 20, loss: 2.1209
Epoch 30, loss: 2.1180
Test accuracy: 0.2949
Features used: 100
Task 8 completed.
```

Task 9 — Evaluation

```
DATASET SIZE COMPARISON
 Training Images  Accuracy  Macro Precision  Macro Recall  Macro F1  Training Time (s)
n Time (s)  Model Size (KB)
           1000    0.3323           0.2499        0.2114    0.2151              0.135
     0.027          2544.90
           5000    0.3846           0.3213        0.2441    0.2541              0.394
     0.026         10093.07
          10000    0.4089           0.3581        0.2658    0.2816              0.793
     0.027         17261.29
          14528    0.4089           0.3528        0.2615    0.2766              1.195
     0.026         23236.73
```